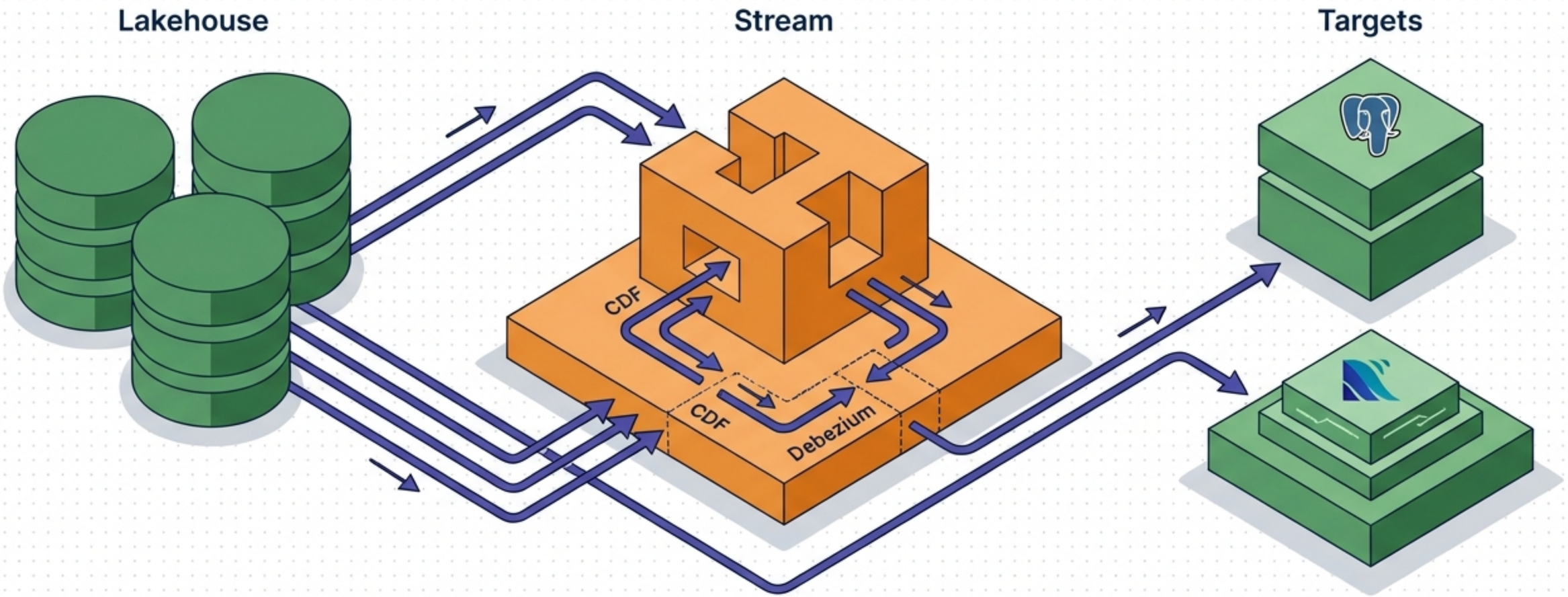


TL;DR Execution Summary

Achieved **Zero-DBU extraction** bypassing Databricks compute via OSS Spark direct-to-storage reads

Built native **Debezium event streaming** directly from Change Data Feed (CDF)

Delivered **Multi-target syncs** (Postgres + RisingWave) from a single Kafka topic, orchestrated by Dagster

Gemini Notebook

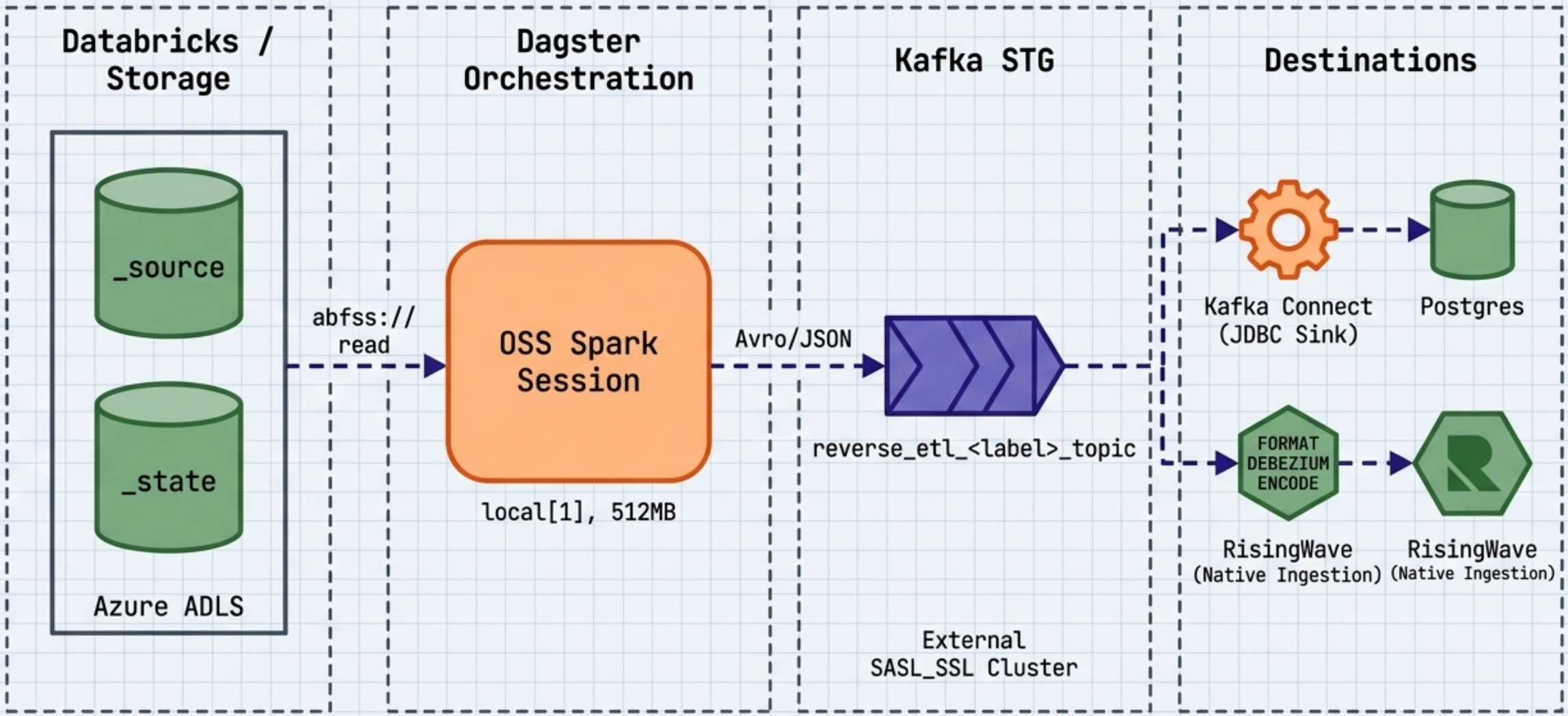
SEMANTIC MISMATCHES AND DESTINATION LIMITS DISQUALIFY EXISTING TOOLS

Evaluation Criteria	drt (v1.0.0)	dlt	Custom Dagster Component
Destination Support	✗ Direct to DB only (No Kafka)	✗ Rejects Kafka destinations	✓ Writes to any Kafka consumer
Extraction Method	✗ Mirror mode (High DBU cost)	✗ Lacks CDF source	✓ Delta Change Data Feed (Pure delta)
Data Semantics	✗ Tracking tables (Loses explicit deletes)	✗ Row-level upserts	✓ Native Debezium CDC envelopes (c/u/d)

! Custom build required for CDC-native extraction, Kafka delivery, and zero-DBU scalability.

Gemini Notebook

End-to-end architecture from Delta storage to downstream consumers



Gemini Notebook

Bypassing Databricks compute drops extraction cost to zero DBUs

The Paid Path

table_changes()

Serverless SQL Warehouse ($$)

The Zero-Cost Path

PySpark 4.0.4 + Delta-Spark

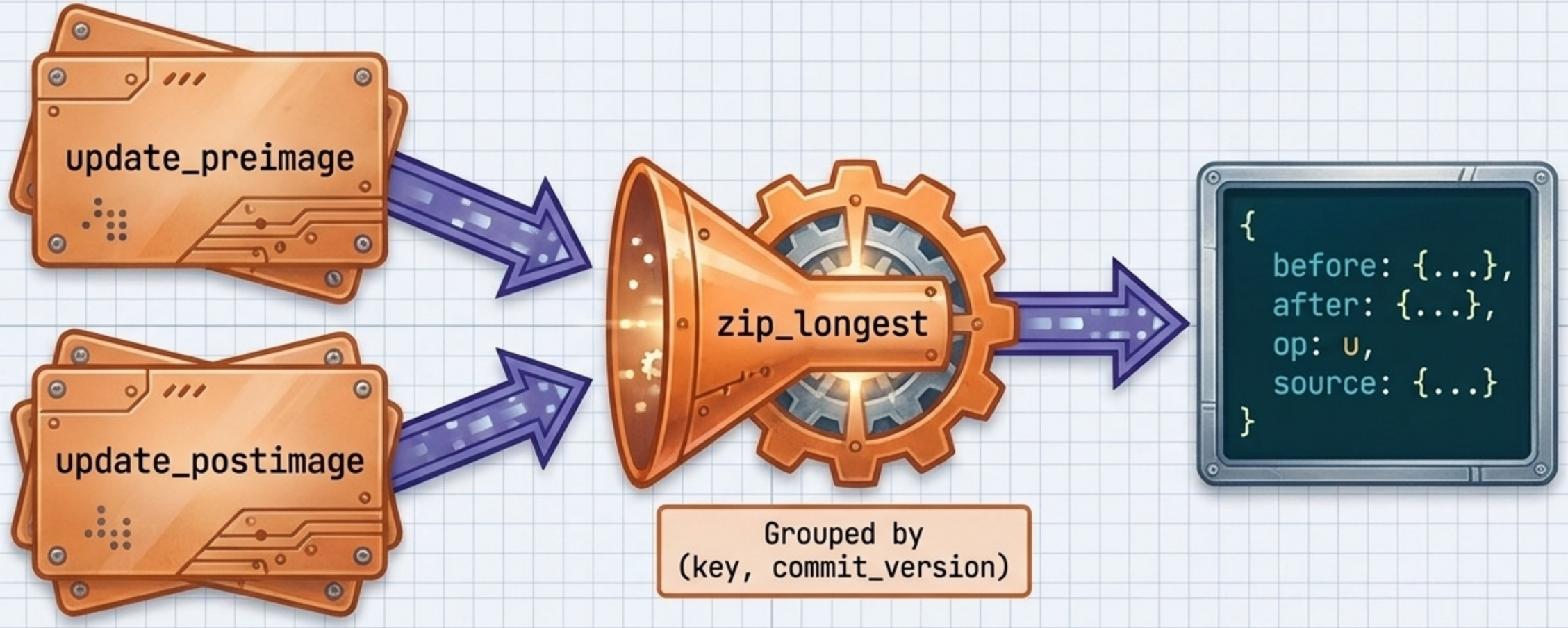
Azure ADLS via ADLS_* credentials (Free)



Architectural Constraint: The __state table must remain plain Delta (not Universal Format) because OSS Spark lacks the IcebergConverter class.

Gemini Notebook

Synthesizing discrete CDF rows into unified Debezium envelopes

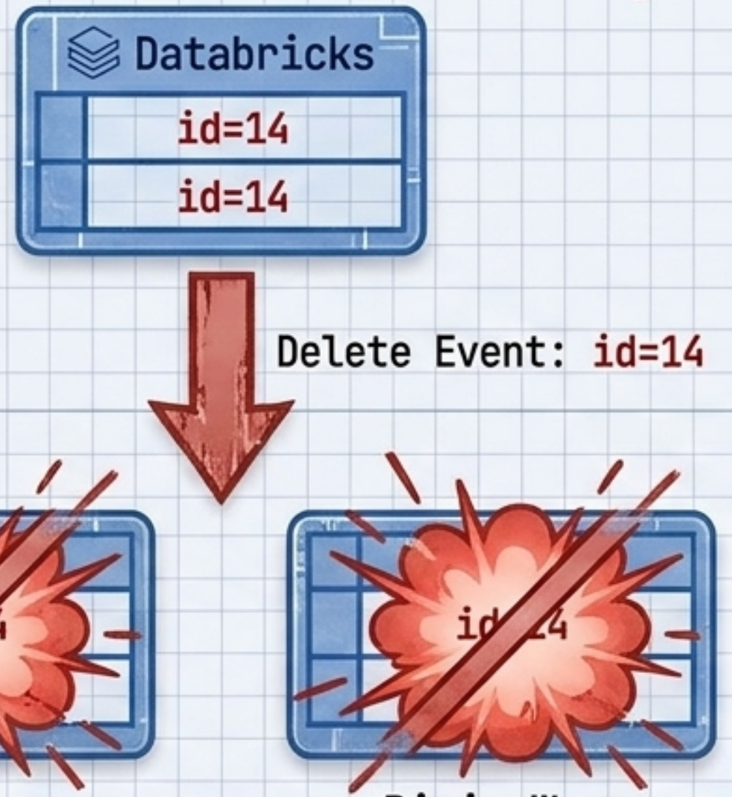


CDF updates produce discrete rows. The pipeline accumulates lists per change type and groups positionally. This safely handles duplicate-keyed source rows mapping to the same commit.

Gemini Notebook

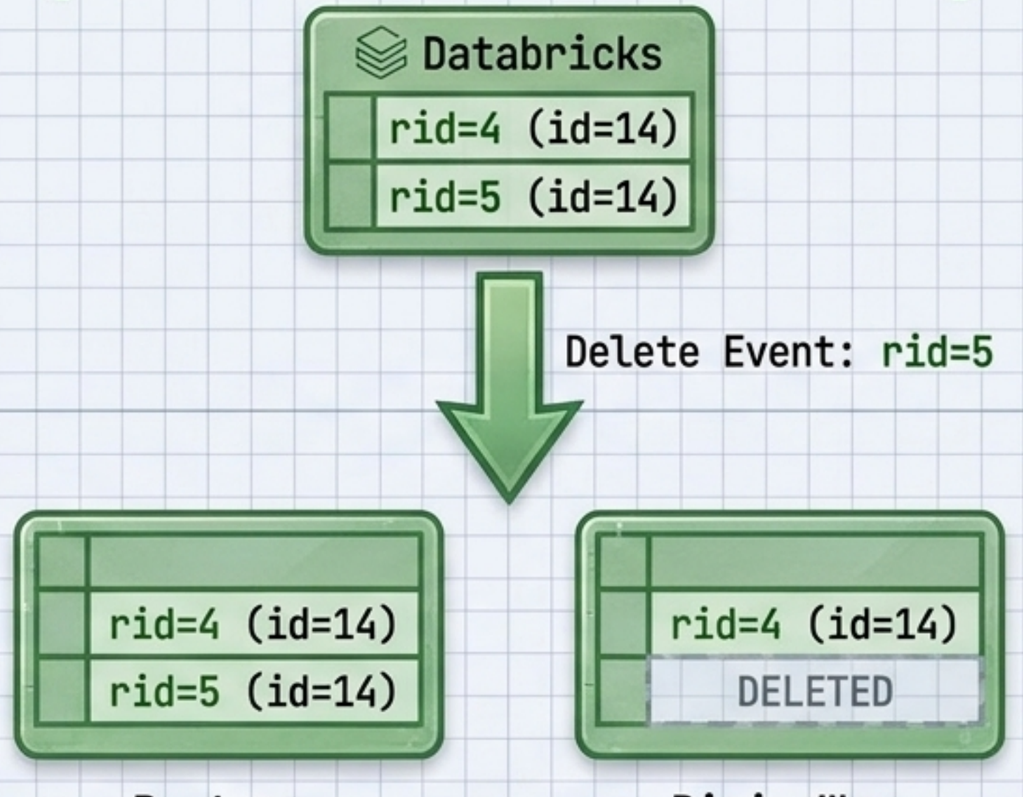
Surrogate identity keys prevent catastrophic delete cascades

The Danger (Business ID)



Relying on business ID causes silent downstream data loss on deletes. Unity Catalog PKs are informational only.

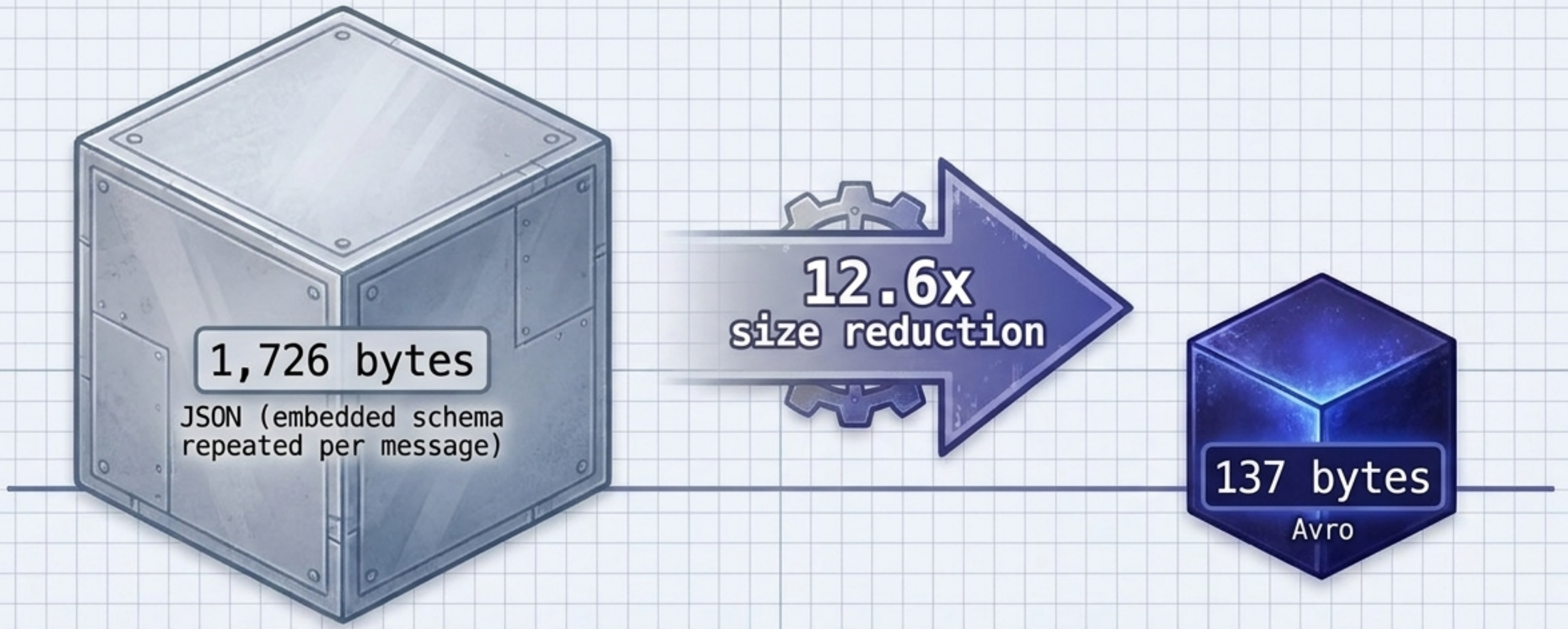
The Solution (Surrogate Key)



All synced tables require rid BIGINT GENERATED ALWAYS AS IDENTITY. This serves as the immutable message key.

Gemini Notebook

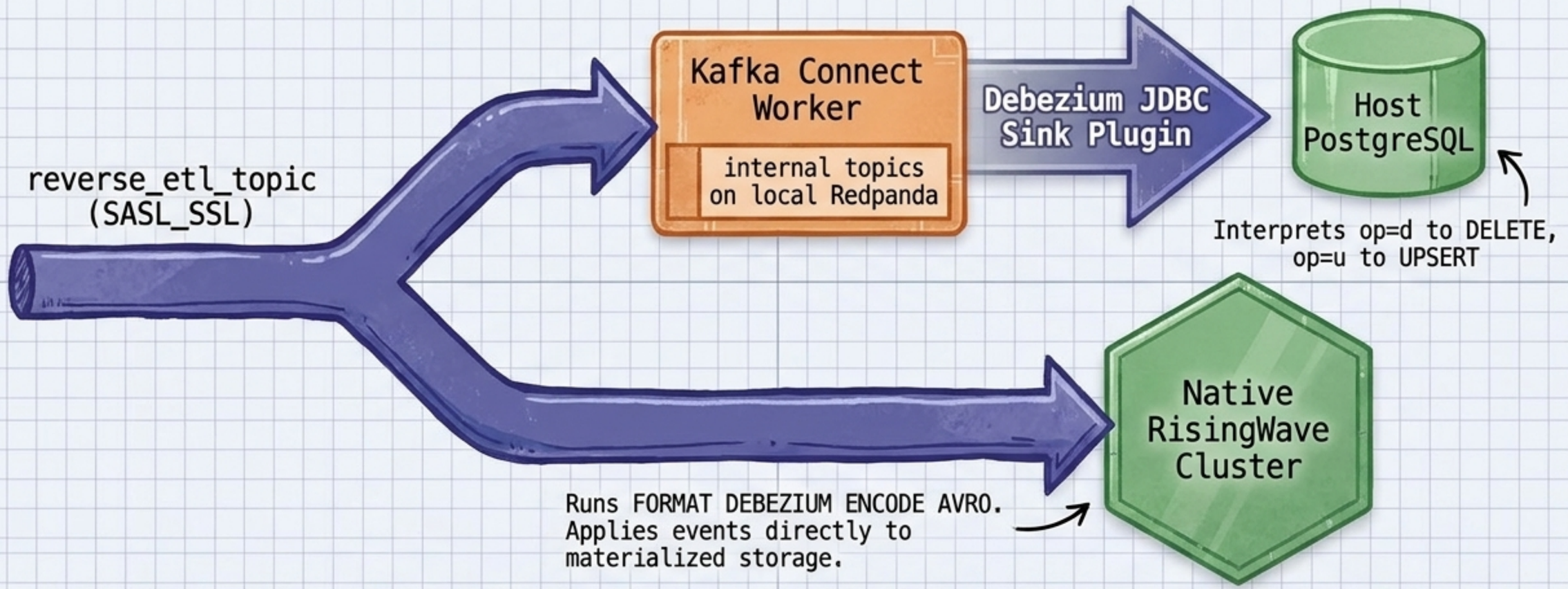
Avro encoding shrinks message payloads by over 90 percent



Requirement:	Constraint:
Shared staging Apicurio registry.	RisingWave requires temporal workaround (timestamp ingested as VARCHAR, generated as TIMESTAMPTZ).

Gemini Notebook

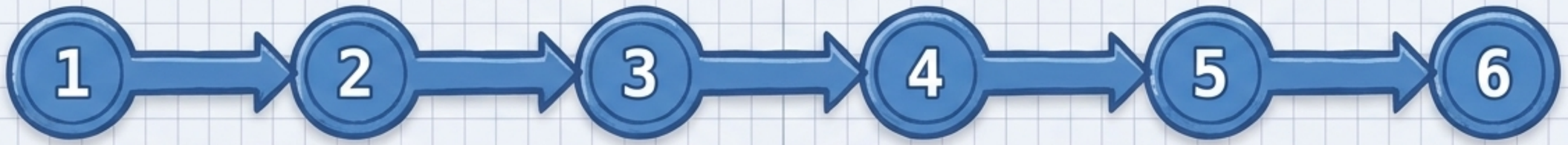
A single Debezium topic drives multiple independent consumers



One Topic, Multiple Readers. Each consumer tracks its own offsets independently.

Gemini Notebook

Orchestrating zero-downtime additive schema evolution



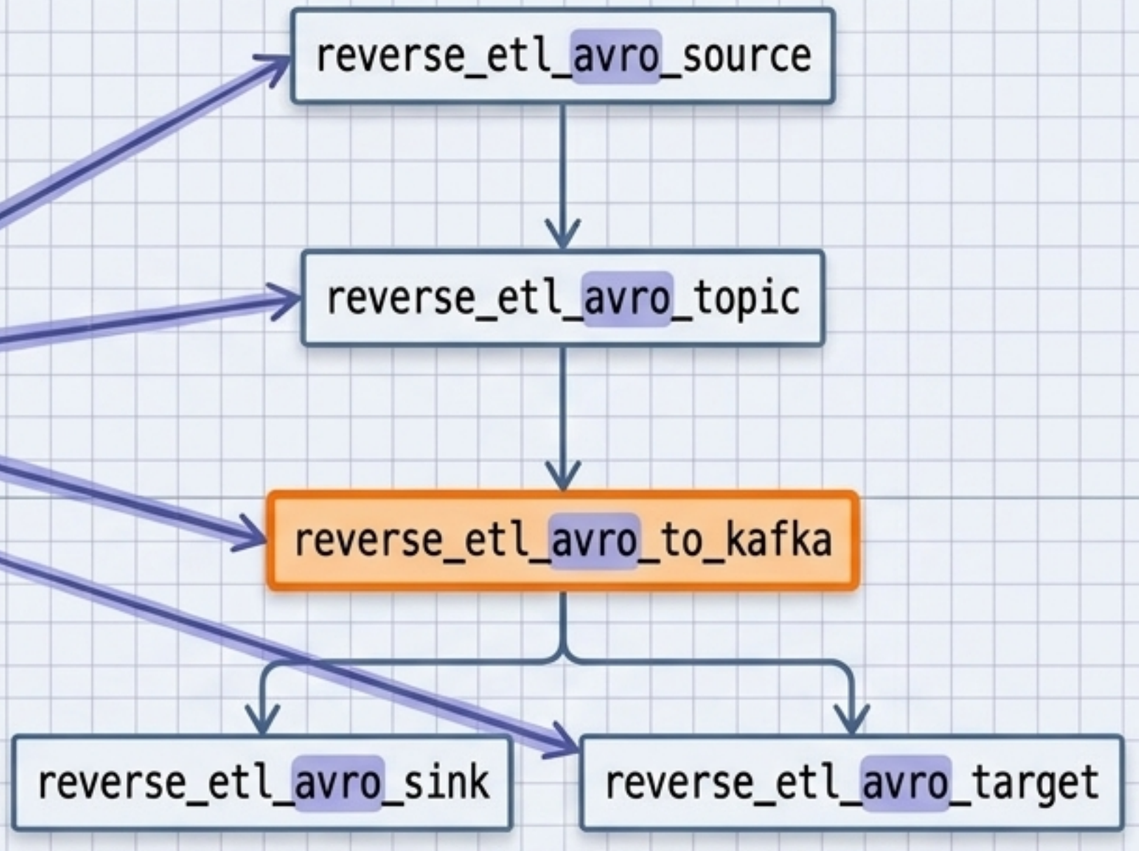
1	2	3	4	5	6
Databricks	Dagster Asset	Dagster Asset	Dagster Asset	Dagster Asset	Postgres Sink
ALTER TABLE... ADD COLUMN	Reads live Delta schema	Registers new schema version to Apicurio Registry	Executes ALTER TABLE on RisingWave	Produces Kafka events	Auto-adds column via schema.evolution=basic

Crucial Constraint: RisingWave must be altered before the first event carrying the new column is produced, otherwise ingested rows default to NULL. Evolution is additive only.

Gemini Notebook

A 10-line YAML config compiles into an isolated execution graph

```
name: avro
extraction: oss_spark
encoding: avro
catalog: de_dev
schema_name: sr_poc_external
```



The ReverseEtlSyncConfig object dynamically generates assets. A single label ensures zero name collisions, allowing isolated parallel syncs.

Gemini Notebook

Operating the pipeline: Setup, Sync, Check, and Reset

Setup

Idempotent IF NOT EXISTS creation of tables and topics.

Sync

Manual or notebook trigger sharing a single Delta watermark.

One trigger per change.

Check

The sink_healthy scheduled asset check.
Prevents silent lag by polling the Connect REST API every 5 minutes.

Reset

Destructive teardown.

Safely isolated via strict reset_name_prefixes allowlists.

Gemini Notebook

Architectural heuristics: When to deploy which pipeline

What is your scale and requirement?

Deploy drt when:

- Handling small/medium reference tables.
- Syncing transformed SQL queries.
- Deletes are not required.
- Targeting a single database destination.
- Simplicity is prioritized.

Deploy Custom Pipeline when:

- Processing billions of rows with millions of daily changes.
- CDC streaming is required (zero-DBU reads).
- Multiple consumers need decoupled access via Kafka.
- Strict schema evolution tracking is mandatory.
- Replayable change history is needed.

Gemini Notebook